

Параметризированное IP-ядро полифазного КИХ-фильтра с конфигурацией ИХ на ходу

Задание 1

1. В Matlab создайте файл Live Scripts (*.mlx).
2. С помощью функций/приложений Matlab создайте КИХ-фильтр нижних частот со следующими характеристиками:
 - 2.1. Порядок $N = 31$;
 - 2.2. $F_s = 20$ МГц;
 - 2.3. $F_{\text{pass}} = 2$ МГц;
 - 2.4. $F_{\text{stop}} = 4$ МГц;
3. Создайте 3 синусоиды, демонстрирующие изменения амплитуды сигнала в полосе пропускания, на частоте среза и в полосе подавления созданного фильтра.
4. С помощью функции `fi` приведите коэффициенты фильтра к формату с фиксированной точкой с длиной слова 16 бит:
 - 4.1. Сначала выполните преобразование по умолчанию – в качестве параметров должны быть указаны только сами значения коэффициентов.
 - 4.2. Выведите двоичное представление полученных чисел с фиксированной точкой, найдите значения с максимальным модулем. Если интерпретировать найденные двоичные представления как целые числа, какие значения соответствуют им?
 - 4.3. Выберите параметры функции `fi` таким образом, чтобы приведение к 16-битному формату с фиксированной точкой происходило с наибольшей точностью.
 - 4.4. Результаты пунктов 4.1. и 4.3. преобразуйте обратно в формат с плавающей точкой и рассчитайте их средний квадрат ошибки

относительно исходных коэффициентов. На основе полученных значений сделайте вывод о сравнительной точности преобразований в пунктах 4.1. и 4.3.

Задание 2

1. В рабочем скрипте Matlab создайте чирп, возрастающий от частоты 1 МГц до 6 МГц длиной 1 мс. Сигнал должен иметь 16-битный целочисленный знаковый формат и максимальную для формата амплитуду.
2. Сравните результаты работы фильтров в двух форматах:
 - 2.1. Проведите фильтрацию сигнала, используя коэффициенты в формате с плавающей точкой, полученных в пункте 2 задания 1.
 - 2.2. Проведите фильтрацию сигнала, используя коэффициенты в формате с фиксированной точкой, полученных в пункте 4 задания 1.
 - 2.3. Вычислите ошибку результата пункта 2.2. по отношению к результату 2.1. Найдите максимальный модуль ошибки и какую долю он составляет от максимального модуля амплитуды сигнала.
 - 2.4. На графике зависимости амплитуды сигнала от времени покажите оба результата фильтрации и ошибку.
 - 2.5. Сравните спектры отфильтрованных сигналов.
 - 2.6. Проведите аналогичный анализ, преобразуя коэффициенты фильтра в 8-битный и 4-х битный форматы. Как изменяются результаты фильтрации при сокращении разрядности?
3. В draw.io (или аналогичном графическом редакторе) создайте RTL-схему элементарного КИХ-фильтра 31-го порядка с коэффициентами в 16-битном знаковом формате, использующего дерево сумматоров и работающего с сигналами в 16-битном знаковом формате:
 - 3.1. Для каждого регистра рассчитайте количество бит, требуемое для хранения промежуточных результатов фильтрации для **произвольного** фильтра и **произвольного** сигнала в указанных выше форматах.

- 3.2. Для каждого регистра рассчитайте количество бит, требуемое для хранения промежуточных результатов фильтрации для фильтра из задания 1 и произвольного сигнала в указанных выше форматах.
- 3.3. Являются ли результаты пунктов 3.1. и 3.2. идентичными? Аргументируйте свой ответ.
4. Создайте аналогичную схему для фильтра в транспонированной архитектуре и проведите идентичный анализ размера регистров.

Задание 3

1. Создайте проект в Vivado, в качестве *part* укажите целевую FPGA.
2. Создайте файл **fir_filter_params_pkg.sv**:
 - 2.1. Файл должен представлять из себя *package* с параметрами создаваемого вами фильтра.
 - 2.2. Все параметры объявляются как *localparam*.
 - 2.3. 3 основных параметра – **SIGNAL_WIDTH**, **COEFF_WIDTH** и **COEFF_NUM**.
 - 2.4. Значения коэффициентов фильтра запишите в массив параметров.
 - 2.5. Все остальные параметры должны рассчитываться на основе вышеперечисленных параметров.
 - 2.6. В качестве ширины промежуточных результатов используйте ширину конечного результата.
 - 2.7. (дополнительно) Для каждого промежуточного результата рассчитайте (на основе вышеперечисленных параметров) минимальную ширину, не приводящую к переполнению при расчётах для вашего фильтра. Запишите результаты в массив параметров. **COEFF_WIDTH** оставьте как 16 бит для каждого коэффициента.
3. Для сигналов используйте тип **logic**.
4. При написании модулей ширина **всех** сигналов (кроме однобитных) должна объявляться с использованием параметров.
5. Регистры, используемые в проекте, должны быть несбрасываемыми.
6. Создайте модуль **filter_cell.sv**:
 - 6.1. Реализуйте параметризуемую ячейку, выполняющую операцию умножения со сложением (MAC).
 - 6.2. Интерфейс модуля:
 - 6.2.1. *clk* (в случае размещения регистров внутри модуля)

- 6.2.2. *i_factor_0* – первый входной множитель.
- 6.2.3. *i_factor_1* – второй входной множитель.
- 6.2.4. *i_addend* – входное слагаемое.
- 6.2.5. *o_result* – выходной результат умножения со сложением.
- 6.3. Ячейка должна представлять из себя самостоятельный модуль, **не** импортирующий **fir_filter_params_pkg**. Список параметров модуля определите на основе требований к модулю.
- 7. Создайте модуль **fir_filter_top**.sv:
 - 7.1. Реализуйте верхний уровень КИХ-фильтра в архитектуре, использующей дерево сумматоров.
 - 7.2. Интерфейс модуля:
 - 7.2.1. *clk* – тактовый сигнал.
 - 7.2.2. *i_signal* – входной отсчёт сигнала.
 - 7.2.3. *o_result* – выходное значение сигнала после фильтрации.
 - 7.3. Модуль должен импортировать параметры из **fir_filter_params_pkg**. Импортируйте сразу все параметры из *package*.
 - 7.4. Операция свёртки должна выполняться с использованием модуля **filter_cell**.
 - 7.5. При написании модуля используйте конструкции **generate** и **genvar**.
 - 7.6. Используйте попарную группировку результатов при реализации дерева регистров.
 - 7.7. (дополнительно) Реализуйте группировку сумматоров на основе параметра – количества слагаемых в одном сумматоре.
 - 7.8. (дополнительно) Для каждого промежуточного регистра используйте параметры из пункта 2.7.
- 8. Запустите стадию elaborate с **fir_filter_top** в качестве *top-level* модуля. Откройте схему получившегося устройства. На основе схемы

рассчитайте задержку фильтрации получения входного отсчёта фильтрации $x[i]$ до выходного результата фильтрации $y[i]$.

9. Создайте файл **fir_filter_tb.sv**:

9.1. Реализуйте простой тестовый сценарий, проверяющий корректность фильтрации, путём сравнения результатов RTL-реализации с результатами целочисленной модели в Matlab.

9.2. Запишите отсчёты исходного и отфильтрованного сигналов в текстовые файлы.

9.3. Модуль тестбенча должен иметь **пустой** интерфейс.

9.4. Период тактового сигнала задайте с помощью параметра. Формирование тактового сигнала реализуйте с помощью конструкции *always*.

9.5. Для управления входными сигналами используйте конструкцию *initial begin*.

9.6. Входной значения должны считываться из соответствующего текстового файла.

9.7. Сравните выходные значения фильтра с результатами целочисленной модели:

9.7.1. Эталонные результаты должны также зачитываться из текстового файла.

9.7.2. Для корректного сравнения используйте значение задержки из пункта 7.

9.7.3. В лог симуляции должно выводиться сообщение о корректности результатов (полное соответствие/расходится хотя бы 1 бит результата).

9.7.4. В случае некорректности результатов, найдите причину ошибки и внесите соответствующие исправления в описания модулей.

10. Добавьте в **fir_filter_top** параметризацию, позволяющую выбрать одну из двух архитектур: с деревом сумматоров и транспонированную:

- 10.1. Добавьте конструкции, позволяющие реализовать вторую архитектуру.
- 10.2. Скорректируйте/дополните **fir_filter_params_pkg** при необходимости.
- 10.3. Повторите пункты 7 и 8 для фильтра в транспонированной архитектуре.